ASIC Implementation of Fetch Unit for a Pipelined RV32 Processor

[1]Deepali Banka, [2]Diya Kurian, [3]Soham Kapur, [4]Dr Augusta Sophy Beulet P
[1,2]*School of Electronics Engineering (SENSE), Vellore Institute of Technology*
[3]*School of Computer Science (SCOPE), Vellore Institute of Technology*
[4]*Centre for Nanoelectronics and VLSI Design (CNVD), Vellore Institute of Technology*
Chennai, India
Corresponding Author email address

***Abstract*— The Fetch Unit of a processor is the first stage in a pipeline which is used to retrieve instructions from the instruction memory as per the desired address. Various processors have been tested on FPGAs for the 32-bit RISC-V Instruction Set Architecture. We have attempted to implement an ASIC for our Fetch Unit in order to lay the foundation for an indigenous processor design. This chip was designed using Verilog HDL and RTL to GDS2 was implemented through the OpenLane library. The block is able to perform the barebones functionalities which is useful for studying and practicing processor stages for academic purposes. The implementation was carried out using open-source tools which makes it suitable for academic and educational purposes. It also makes the design economical for research purposes.**

Keywords*— *RISC-V, Fetch Cycle, OpenLane, ASIC, Pipelined

I. INTRODUCTION

In this research paper, we have focused on the design and implementation of a 32-bit Instruction-Set Fetch Unit of a RISC-V processor till physical design. The Fetch Unit has been successfully designed to obtain instructions sequentially from a designated Instruction Memory, as per the Harvard Architecture. We aim to showcase the capabilities of a Fetch Unit in handling standard operations like Branch and Jump on its own, so that these operations need not be done by the Execute Unit nor have the need for a dedicated branch predictor. Our purpose behind adding a compact Fetch Unit was to help increase the performance of the processor as a whole (Tiwari et al, 2023).

The most appropriate language for designing the Fetch Unit is Verilog programming language, which is widely preferred for hardware development (Tan and Rosdi, 2014). The synthesis is done using the open source Yosys tool, which gives a reliable output (Wolf et al, 2013). In addition to this, the physical design incorporated the OpenRoad library, through the OpenLane flow. This is a open source tool for RTL to GDS2 flow that works on Python libraries. It can be executed through a Linux-based environment or an online environment also (Ghazi, 2020; Hesham, 2021; Chupilko, 2021). The open-source 130 nanometer PDK SKY130A was used for physical design, which is useful for research as well as commercial purposes (Shalan, 2020).

The Fetch Unit is the primary interface between the processor and the Random Access Memory (RAM). It is written to ensure systematic reading of instructions from the memory that need to be performed within a single clock cycle in case of RISC based processors (Samanta et al, 2024). In the design of the Fetch Stage, modularity is essential to be able to introduce new features during future developments. Interfacing with multiple memory blocks in case of parallel processing, or adding a cache to store frequently used instructions should be possible without any major changes to the rest of the design (Saussereau et al, 2023). Despite being evident that FPGA is useful for prototyping and testing of the processor (Raveendran, 2016 and Poli, 2021), our motive behind approach towards ASIC implementation is to help optimize the design performance.

II. METHODOLOGY

A. Fetch Unit Architecture

The Fetch Unit is designed to perform incremental, Branch and Jump operations. It can access the instructions stored in the instruction memory and display it as per the desired address, which may be given by the Execute Unit also.

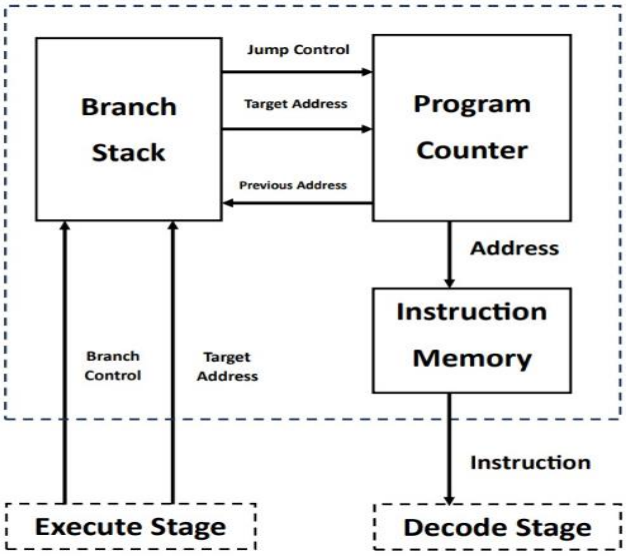

Fig. 1. Fetch Unit Architecture

The Fetch Unit is designed in such a way that it minimizes the number of additional wires required for Branch, Jump and Stall operations. The Branch operation works with a stack to ensure swift transference to and from the subroutine based on a 2-bit signal. The data transfer follows the Parallel-In-Parallel-Out (PIPO) method, which helps maintain high speed of rations. Out of the multiple modules, only a single module contains the clock signal and the rest are combinational circuits, triggered by this module.

B. Instruction Memory Architecture

The Instruction Memory (IMem) consists of a 5-bit input line, which supplies the instruction address, and a 32-bit output line, which gives the Instruction at that address to the next unit. Both the lines are connected to PIPO registers to store their data until the next clock cycle There is a Reset line,

which sets the Address register back to zero. This is useful for Flush operations.

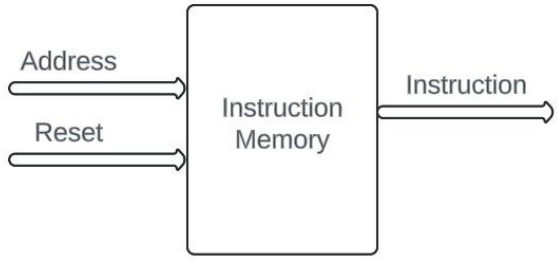

Fig. 2. Instruction Memory Architecture

The Instruction Memory module is the module that houses the register bank for storing the list of instructions to be executed by the processor. It takes the desired Address as the input and outputs the Instruction stored at that address. The Reset line, when activated, outputs no instruction. Hence the Instruction Register is also reset.

C. Branch Stack Architecture

The Branch operation is one where the sequential reading of instructions in a given routine is broken intentionally to access another subroutine, after completing which the control returns back to the main routine. The Jump operation is a command where the control jumps to another line of the code, but is not expected to return to the same routine.

The Branch Stack functional unit consists of a data path, control path and a register stack. The control path sends the control signals Branch and Reset to decide the operation on the stack counter. The data path has the Target address as input, which provides the address which needs to be pushed to the stack if the operation is 'push'. The Pop outputs the last address on the stack if the operation is 'pop'.

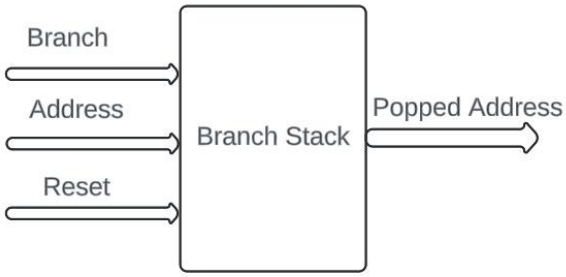

Fig. 3. Branch Stack Architecture

The stack stores the addresses pushed in chronological order, and helps retrieve the previous line from where the branch was started. In our code the branch stack module is a hardware implementation of a stack data structure. When the control asks for a branch action, the current address is read from the Program Counter and pushed to the stack. When the control asks to exit a branch, the last address in the stack is popped and sent to the Program Counter.

D. Program Counter

The Central Processing Unit (CPU) of a computer contains a register called the Program Counter (PC), which is also often called the Instruction Pointer. Its purpose is to show the user where the computer is in its sequence of instructions. Essentially, it preserves (or 'counts') the memory location of the active instruction and indicates the subsequent one. The CPU receives the instruction from memory that the Program Counter points to during the fetch cycle.

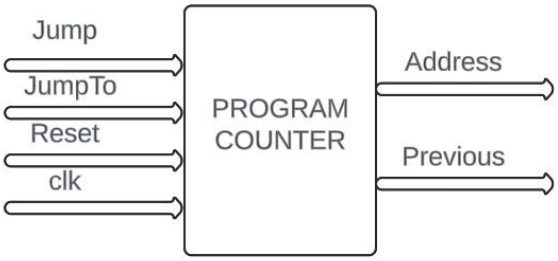

Fig. 4. Program Counter Architecture

In our design the Program Counter module is the module responsible for periodically updating the address of the instruction to be read. It is the only module in the Fetch Stage that is connected to the Clock signal. The Reset line resets the Address register to Zero. The Jump line decides if there is a different address to jump to, breaking the sequence. The JumpTo line gives the address to which the Program Counter must jump. Once updated, the PC counts from the new address.

III. Results and Discussion

The ASIC implementation of the design was carried out using open-source tools available through the OpenLane workflow. The resulting floorplan had the dimensions as Die Area of **105.625 x 116.345** μm^2 and Core Area of **99.82 x 103.36** μm^2.

The Static Timing Analysis resulted in no negative slack. The circuit contains 415 device instances, and 413 nets. The Design Rule Checks (DRC) and Layout Versus Schematic (LVS) were successfully carried out and the design was found to have no errors, with the layout and schematic circuits matching uniquely.

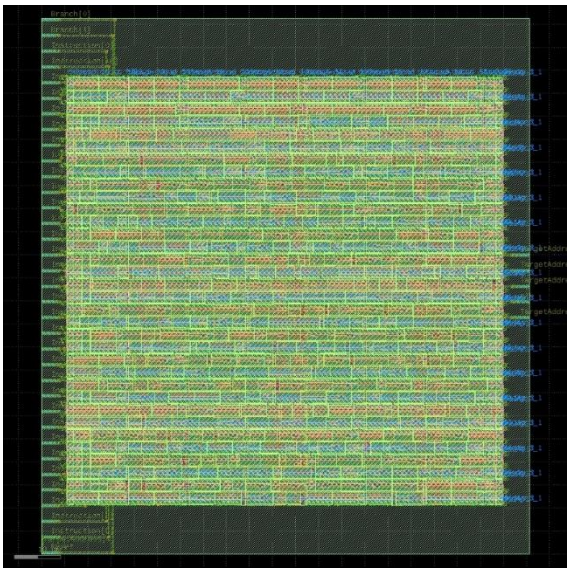

Fig. 5. Physical Design of the Fetch Unit

IV. Conclusion

In this design, we have successfully created the Fetch stage of a pipelined 32-bit RISC-V instruction set processor. This design was completed from RTL till the GDS2 flow. It provides all basic functionality of a Fetch Unit need for a barebones processor, by not applying a branch predictor. It has an expandable Instruction Memory, which is currently up to 4 kilobytes. If connected to an external RAM, this can be repurposed as a cache memory to speed up the pipelining process.

Acknowledgment

The authors would like to express their gratitude to the Centre for Nanoelectronics and VLSI Design, VIT Chennai for their support and guidance in carrying out this project.

References

[1] Tiwari, Ankita, et al. "IndiRA: Design and Implementation of a Pipelined RISC-V Processor." 2023 33rd International Conference Radioelektronika (RADIOELEKTRONIKA). IEEE, 2023.

[2] Raveendran, Aneesh, et al. "A RISC-V instruction set processor-micro-architecture design and analysis." 2016 International Conference on VLSI Systems, Architectures, Technology and Applications (VLSI-SATA). IEEE, 2016.

[3] Saussereau, Jonathan, et al. "Design and Implementation of a RISC-V core with a Flexible Pipeline for Design Space Exploration." 2023 30th IEEE International Conference on Electronics, Circuits and Systems (ICECS). IEEE, 2023.

[4] Poli, Ludovico, et al. "Design and Implementation of a RISC V Processor on FPGA." 2021 17th International Conference on Mobility, Sensing and Networking (MSN). IEEE, 2021

[5] Tan, Tze Sin, and Bakhtiar Affendi Rosdi. "Verilog hdl simulator technology: a survey." Journal of Electronic Testing 30 (2014): 255-269.

[6] Wolf, Clifford, Johann Glaser, and Johannes Kepler. "Yosys-a free Verilog synthesis suite." Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip). Vol. 97. 2013.

[7] Ghazy, Ahmed, and Mohamed Shalan. "Openlane: The open-source digital asic implementation flow." Proc. Workshop on Open-Source EDA Technol.(WOSET). 2020.

[8] Hesham, Sarah, et al. "Digital ASIC Implementation of RISC-V: OpenLane and Commercial Approaches in Comparison." 2021 IEEE International Midwest Symposium on Circuits and Systems (MWSCAS). IEEE, 2021.

[9] Chupilko, Mikhail, Alexander Kamkin, and Sergey Smolov. "Survey of Open-source Flows for Digital Hardware Design." 2021 Ivannikov Memorial Workshop (IVMEM). IEEE, 2021.

[10] Shalan, Mohamed, and Tim Edwards. "Building OpenLANE: A 130nm openroad-based tapeout-proven flow." Proceedings of the 39th International Conference on Computer-Aided Design. 2020.

[11] Höller, Roland, et al. "Open-source risc-v processor ip cores for fpgas—overview and evaluation." 2019 8th Mediterranean Conference on Embedded Computing (MECO). IEEE, 2019.

[12] Samanta, Anu, et al. "A Comprehensive Survey and Comparison on Pipelined RISC System Architectures." Journal of Electrical Systems 20.3 (2024): 2817-2825.